PxD PRECISION DEVELOPMENT

July 2026

Digital Agriculture Advisory Based Recruitment Exercise

Estimated Completion Time

2–2.5 hours

This assessment evaluates your ability to:

- Understand farmer segmentation and select appropriate digital advisory channels.
- Design a digital agriculture advisory system architecture.
- Build a simple technical prototype demonstrating how advisory content can be delivered through digital channels.

Allowed Tools

You may use the following tools during the assessment:

1. Spreadsheet Tool (***Question 1***): Google Sheets or Excel Online for the farmer profiling task.
2. Diagram Tool (***Question 2***)
   - draw.io or Excalidraw Whiteboard for the architecture sketch.
   - Both tools are browser-based and do not require installation or account registration.
   - We only need your final diagram and explanation.
3. Writing Tool: Any plain text editor or word processor (Notepad, Word, Google Docs, etc.) for written responses.

4. Web Search: General web search is allowed.
5. AI Tools:
   - AI tools such as ChatGPT, Claude, or similar tools are allowed for drafting, brainstorming, or technical assistance.
   - However, your final answers must demonstrate your own understanding and reasoning, especially in the short-answer sections.

You do not need to install any software locally for this assessment.

6. Environment Setup (***Question 3***) (API + Docker)

For Question 3, use one of the following free browser-based environments:

Option 1: GitHub Codespaces

1. Create a free GitHub account if you do not already have one.
2. Create a new repository.

3. Click: Code → Codespaces → Create codespace
4. This provides a complete browser-based development environment with a terminal and Docker support.

Option 2: Play with Docker

1. Visit Play with Docker, labs.play-with-docker.com
2. Log in using a free Docker Hub account.
3. Click "Add New Instance" to start a Docker-enabled terminal environment.

Either option is acceptable. Use whichever environment you are more comfortable with.

Submission Requirements

Submit all materials together in one email or one document.

Question 1 Submission

Submit:

- One shared Google Sheet or Excel file link containing your farmer profiling assessment.

The sheet should include your assessment of all 10 farmers with:

- Recommended delivery channel(s):
  - SMS
  - IVR
  - Video
  - Combination of channels
- One-sentence reasoning explaining your recommendation.

Question 2 Submission

Submit:

- Exported architecture diagram:
  - PNG
  - PDF
  - OR shareable draw.io/Excalidraw link
- Written explanation below your diagram (5–8 sentences).

Question 3 Submission

Submit:

- GitHub repository link OR uploaded project files containing:
  - API source code
  - Dockerfile
  - README file
- Evidence that the API runs successfully:
  - Terminal output showing the API running successfully
  - Screenshot
  - OR accessible running endpoint link

Background Context

Centralized Advisory for Ethiopia

Centralized Advisory for Ethiopia is a digital agriculture advisory service that produces two types of agricultural content:

1. An AI-based weather advisory short-term forecasts translated into planting/irrigation guidance (e.g., the rain season will start in 15 days, please do the required preparation in advance “ or "rain expected in 2 days, delay fertilizer application")
2. A fertilizer advisory recommendations on what to apply, how much, and when, based on crop stage and soil/region data

This content needs to reach farmers through IVR (voice), SMS (text), and video, without the content team recreating each advisory three separate times.

Farmers differ in device access:

- Farmer Group A personal Android smartphone
- Farmer Group B basic/button feature phone
- Farmer Group C no personal phone; shares one feature phone with the household

Question 1 Farmer Profiling

You are given 10 sample farmer profiles below (device type, ownership, literacy rate, region/notes). For each farmer, assign the best delivery channel SMS, IVR, video, or a combination and justify your choice in one sentence.

	Device	**Ownership**	**Read SMS comfortably?**	**Notes**
1	Android smartphone	Personal	Yes	Good data connection
2	Feature phone	Personal	Yes	
3	Feature phone	Shared with family	No	
4	Android smartphone	Personal	No	Low literacy, owns smartphone via son
5	Feature phone	Personal	No	
6	No phone	Shares neighbor's phone occasionally	No	
7	Android smartphone	Shared with household	Yes	Poor network in region

8	Feature phone	Personal	Yes	Prefers voice over text anyways
9	Android smartphone	Personal	Yes	
10	Feature phone	Shared with family	Yes	

Question 2 Single Knowledge Base Architecture

Using draw.io or Excalidraw, sketch a simple architecture (boxes and arrows) for Centralized Advisory for Ethiopia showing:

- One place where both advisory types (weather, fertilizer) are authored/generated once
- Where the AI component sits specifically for the weather advisory (e.g., pulling a weather forecast API/model and generating a plain-language recommendation) — versus the fertilizer advisory, which may be more rule-based/static
- A channel adaptation layer that turns either advisory into an SMS text, an IVR voice script, and a short video, without duplicating content creation.
- Where the farmer profile (from Question 1) determines which channel(s) a given farmer actually receives.
- One external dependency the system would realistically need (e.g., weather forecast API/model, text-to-speech engine, video hosting/streaming service)

Below the diagram, write a short paragraph (5–8 sentences) explaining:

- One assumption you made about data flow
- One risk or limitation of your design (e.g., forecast uncertainty in the AI weather advisory, low connectivity, shared-phone privacy)
- One thing you would ask a technical/dev team before finalizing this design.

Question 3 — Basic API Setup & Dockerization

This continues directly from your Question 2 architecture: you sketched a **Channel Adaptation Layer** that turns one advisory into SMS/IVR/video outputs.

Now build the smallest possible working piece of that idea, entirely in a free, browser-based environment (GitHub Codespaces or Play with Docker see Environment Setup above). No local installation is required or expected.

Task:

1. Build **one simple REST API endpoint** (any free, open-source framework — e.g., Python Flask/FastAPI, or Node.js/Express) that:
    - Accepts a farmer_id or region as a parameter (e.g., GET /advisory?region=Bahir%20Dar)

- Returns a **mock advisory** as JSON — you can hardcode 2–3 sample responses (one weather advisory, one fertilizer advisory); no real weather API integration needed
- Example response:

json

```
JSON
     {
       "region": "Bahir Dar",
       "type": "weather",
       "message": "Rain expected in 2 days. Delay fertilizer
application.",
       "confidence": "medium"
     }
```

2. **Containerize the API** using Docker, inside your Codespace or Play with Docker session:
   - Write a Dockerfile that builds and runs your API
   - The API should be reachable on a port (e.g., 5000) once the container is running — Codespaces will auto-forward the port with a clickable link; Play with Docker shows an "OPEN PORT" button

3. **Write a short README** (half a page) covering:
   - How to build and run the container (exact commands, e.g., docker build -t advisory-api . and docker run -p 5000:5000 advisory-api)
   - How you tested it (a curl command run in the Codespaces/Play with Docker terminal, or a Postman Web example, is enough)
   - One sentence on how this single API endpoint would plug into the channel adaptation layer from your Question 2 diagram (i.e., what would call this API — the SMS formatter? the
   - IVR/TTS engine?)